

06. measure ROS2 launch 入口

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	5
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 launch Action、Node Action、実行可能名、remapping	6
6.7 ROS graph、Node、topic、service、action、parameter	7
6.8 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか	7
7 関数・クラスを上から順に解説	8
7.1 L11 関数 _cfg_path	8
7.2 L15 関数 _setup	8
7.3 L97 関数 generate_launch_description	10
7.4 launch から起動するノード	11
8 処理の流れ	11

1 対象

項目	内容
ファイル	launch/solar_measurement.launch.py
ソース SHA-256	070ba41b225288b85a317c3c102783d85d9df6fe189953a54ec69bf327569752
種別	ROS 2 launch Python
区分	launch
役割	実測収集用に preflight、distance、grade、logger、dashboard を起動する launch。
起動文脈	measure モード起動時に ros2 launch される。

2 このファイルを読む理由

実測収集用に preflight、distance、grade、logger、dashboard を起動する launch。

- planner を起動せず、計測系だけを立てる。
- distance/grade は profile の measurement 設定で可否が決まる。

3 呼び出し関係

3.1 どこから呼ばれるか

- scripts/solar_control.sh
- SolarSim.ps1

3.2 次に読むと理解が進むファイル

- mpc_solarcar/distance_node.py
- mpc_solarcar/grade_node.py
- mpc_solarcar/solar_logger_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
grade_cfg = get_section(measurement_cfg, "grade")
meas_logging_cfg = get_section(measurement_cfg, "logging")

nodes = [
    Node(
```

```

package="mpc_solarcar",
executable="solar_preflight_node",
name="solar_preflight_node",
parameters=[
    {
        "require_speed": True,
        "require_distance": bool(measurement_cfg.get("use_distance_node", True))
    },
    {
        "require_battery": False,
        "require_planner": False,
    }
]

```

5 主要構造の読み取り

主要関数は generate_launch_description。launch から起動する Node action は 5 件。

5.1 主要クラス・関数

行	種別	名前	補足
11	function	_cfg_path	args: profile_path, raw
15	function	_setup	args: context
97	function	generate_launch_description	

5.2 ファイルを上から読んだときの定義順

行	内容
11	関数 _cfg_path を定義する。
15	関数 _setup を定義する。
97	関数 generate_launch_description を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	from __future__ _import annotations	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	from launch import LaunchDescription	launch が実行すべき action 群をまとめるため。このファイル内での主な使用位置は L98。
4	from launch. actions import DeclareLaunchArgument, OpaqueFunction	DeclareLaunchArgument や OpaqueFunction など launch action を使うため。このファイル内での主な使用位置は L100, L101。
5	from launch. substitutions import LaunchConfiguration	launch 引数の実行時値を参照するため。このファイル内での主な使用位置は L16。
6	from launch_ros. actions import Node	ROS2 ノード起動 action を launch から記述するため。このファイル内での主な使用位置は L26, L39, L50, L65, L79。

8	frommpc_solarcar.solar_profileimportget_section, load_profile, resolve_relative_path	profile YAML 読込と検証から <code>get_section</code> , <code>load_profile</code> , <code>resolve_relative_path</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L12, L17, L18, L19, L20, L21, L22, L23。
---	--	---

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘`x = f()`’では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

‘`float(...)`’、‘`int(...)`’、‘`bool(...)`’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘`None`’は値が無いことを示す単一のオブジェクト、‘`math.nan`’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘`is None`’と ‘`math.isfinite`’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘`def`’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘`f`’は関数そのもの、‘`f()`’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘`*args`’、‘`**kwargs`’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが `'self.z'` を更新すればオブジェクトの状態が残るが、単に `'z'` を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。`'from .model import SolarCarModel'` の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。`'def'` や `'class'` は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の `'console_scripts'` は、端末で使う実行可能名と `'package.module:function'` を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。`'try/except'` は想定した異常を処理し、`'finally'` は成功・失敗にかかわらず後始末を行う。

`'with open(...) as f:'` は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

`'except Exception: pass'` はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 launch Action、Node Action、実行可能名、remapping

`'launch_ros.actions.Node(...)'` は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

`'DeclareLaunchArgument'` は launch 実行時に受け取る入力欄を宣言し、`'LaunchConfiguration'` はその値を後で解決する substitution を表す。`'perform(context)'` は実行時 context から確定文字列を取り出す。

launch の `'name'` は Node 名 override、`'parameters'` は起動時 parameter、`'remappings'` は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

6.7 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.8 rqt_graph、ros2 CLI、rosbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

7 関数・クラスを上から順に解説

7.1 L11 関数 `_cfg_path`

- 行範囲: L11–L12
- 所属: module 直下
- 定義: `_cfg_path(profile_path, raw:str) -> str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `resolve_relative_path`, `str`, `strip`
- 戻り値の要点: `resolve_relative_path(profile_path.parent, raw) if str(raw or "") else ""`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `resolve_relative_path(profile_path.parent, raw) if str(raw or "") else ""` を返す。

代表コード断片

```
def _cfg_path(profile_path, raw: str) -> str:  
    return resolve_relative_path(profile_path.parent, raw) if str(raw or "").strip() else ""
```

7.2 L15 関数 `_setup`

- 行範囲: L15–L94
- 所属: module 直下
- 定義: `_setup(context)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `LaunchConfiguration`, `Node`, `_cfg_path`, `append`, `bool`, `float`, `get`, `get_section`, `int`, `load_profile`, `perform`, `str`
- 戻り値の要点: `nodes`
- 主なローカル代入名: `cfg`, `distance_cfg`, `grade_cfg`, `logging_cfg`, `meas_logging_cfg`, `measurement_cfg`, `nodes`, `profile_path`, `profile_yaml`, `runtime_cfg`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. profile_yaml に LaunchConfiguration('profile_yaml').perform(context) の結果を代入する。
2. (profile_path, cfg) に load_profile(profile_yaml) の結果を代入する。
3. runtime_cfg に get_section(cfg, 'runtime') の結果を代入する。
4. logging_cfg に get_section(cfg, 'logging') の結果を代入する。
5. measurement_cfg に get_section(cfg, 'measurement') の結果を代入する。
6. distance_cfg に get_section(measurement_cfg, 'distance') の結果を代入する。
7. grade_cfg に get_section(measurement_cfg, 'grade') の結果を代入する。
8. meas_logging_cfg に get_section(measurement_cfg, 'logging') の結果を代入する。
9. nodes に [Node(package='mpc_solarcar', executable='solar_preflight_node', name='solar_preflight_node', parameters=[{'require_speed': True, 'require_distance': bool(measurement_cfg.get('use_distance_node', True)), 'require_battery': False, 'require_planner': False}], Node(package='mpc_solarcar', executable='dashboard_node', name='dashboard_node', parameters=[{'host': str(runtime_cfg.get('dashboard_host', '0.0.0.0')), 'port': int(runtime_cfg.get('dashboard_port', 8080))}], Node(package='mpc_solarcar', executable='solar_logger_node', name='solar_logger_node', parameters=[{'log_dir': _cfg_path(profile_path, str(logging_cfg.get('log_dir', 'outputs/logs'))), 'file_prefix': str(meas_logging_cfg.get('file_prefix', 'solar_measurement')), 'log_rate_hz': float(logging_cfg.get('log_rate_hz', 2.0))}])] の結果を代入する。
10. 条件 bool(measurement_cfg.get('use_distance_node', True)) を判定し、真なら内部処理を行う。
11. nodes.append(...) を実行する。
12. 条件 bool(measurement_cfg.get('use_grade_node', True)) を判定し、真なら内部処理を行う。
13. nodes.append(...) を実行する。
14. nodes を返す。

代表コード断片

```
def _setup(context):
    profile_yaml = LaunchConfiguration("profile_yaml").perform(context)
    profile_path, cfg = load_profile(profile_yaml)
    runtime_cfg = get_section(cfg, "runtime")
    logging_cfg = get_section(cfg, "logging")
    measurement_cfg = get_section(cfg, "measurement")
    distance_cfg = get_section(measurement_cfg, "distance")
    grade_cfg = get_section(measurement_cfg, "grade")
    meas_logging_cfg = get_section(measurement_cfg, "logging")

    nodes = [
        Node(
            package="mpc_solarcar",
            executable="solar_preflight_node",
            name="solar_preflight_node",
            parameters=[
                {
                    "require_speed": True,
                    "require_distance": bool(measurement_cfg.get("use_distance_node", True))
                }
            ],
            
```

```

        "require_battery": False,
        "require_planner": False,
    },
],
),
Node(
    package="mpc_solarcar",
    executable="dashboard_node",
    name="dashboard_node",
    parameters=[
        {
            "host": str(runtime_cfg.get("dashboard_host", "0.0.0.0")),
            "port": int(runtime_cfg.get("dashboard_port", 8080)),
        }
    ],
),
...

```

7.3 L97 関数 generate_launch_description

- 行範囲: L97–L103
- 所属: module 直下
- 定義: generate_launch_description()
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: DeclareLaunchArgument, LaunchDescription, OpaqueFunction
- 戻り値の要点: LaunchDescription([DeclareLaunchArgument('profile_yaml', default_value='config/solar/bwsc_2027_demo.yaml'), OpaqueFunction(function=_setup)])
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. LaunchDescription([DeclareLaunchArgument('profile_yaml', default_value='config/solar/bwsc_2027_demo.yaml'), OpaqueFunction(function=_setup)]) を返す。

代表コード断片

```
def generate_launch_description():
    return LaunchDescription(
        [
            DeclareLaunchArgument("profile_yaml", default_value="config/solar/bwsc_2027_demo
.yaml"),
            OpaqueFunction(function=_setup),
        ]
    )
```

7.4 launch から起動するノード

行	executable	package / name
26	solar_preflight_node	package=mpc_solarcar, name=solar_preflight_node
39	dashboard_node	package=mpc_solarcar, name=dashboard_node
50	solar_logger_node	package=mpc_solarcar, name=solar_logger_node
65	distance_node	package=mpc_solarcar, name=distance_node
79	grade_node	package=mpc_solarcar, name=grade_node

8 処理の流れ

1. launch 引数を受け取り、profile を読み込む。
2. Node action を動的に組み立てる。
3. LaunchDescription として ROS 2 launch へ返す。

9 このファイルの位置づけ

この資料群では、`launch/solar_measurement.launch.py` を **launch** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。